

Sheets



Sheet 1: Core CS Subjects (DBMS, OOPS, CN, OS)

🎯 Goal → Be strong in these since **all product-based interviews** test at least 1–2 core subjects.

Subject	Key Concepts (must know)	Importance Level	Revision Frequency	Notes
OOPS	Encapsulation, Inheritance, Polymorphism, Abstraction; Constructor/Destructor; Access Specifiers; Interfaces vs Abstract Classes; Overloading vs Overriding; Static vs Dynamic binding; Design Patterns (Singleton, Factory, Observer)	★★★★★ (Very High)	Weekly	Interviewers often start with OOPS → most tricky part is real-world examples & implementation in Java .
DBMS	ER diagrams, Normalization (1NF–3NF), Keys (Primary, Foreign, Candidate, Composite), Joins (Inner, Left, Right, Full), Transactions (ACID), Indexing, Views, Stored Procedures, Deadlocks	★★★★★	Weekly	Very common in interviews. Focus on Joins & Transactions .
CN	OSI vs TCP/IP layers, IP vs MAC, TCP vs UDP, 3-way handshake, Routing basics, DNS, HTTP vs HTTPS, Cookies vs Sessions	★★★★★	Bi-weekly	Not too deep. They usually ask TCP vs UDP, OSI model basics .
OS	Processes vs Threads, Scheduling (FCFS, SJF, Round Robin), Deadlocks (necessary conditions, prevention),	★★★★★	Bi-weekly	Focus on deadlocks, scheduling, and process vs

Subject	Key Concepts (must know)	Importance Level	Revision Frequency	Notes
	Memory management (Paging, Segmentation, Virtual Memory), Synchronization (Semaphore, Mutex)			thread differences.

Strategy:

- Revise **OOPS + DBMS every week** (high frequency).
- CN + OS every **2 weeks**.
- Keep **short notes** of definitions, diagrams, and differences (these are fav questions).



Sheet 2: System Design (for your level)

 **Goal** → Not expected to master, but to **defend projects with confidence** and show awareness.

Topic	Key Concepts (must know)	Importance	Revision Frequency	Notes
Scalability basics	Vertical vs Horizontal scaling, Load Balancer, CDN	★★★★★	Monthly	Helps when asked "What if 1M users use your project?"
Database	SQL vs NoSQL, Indexing, Partitioning, Sharding, Replication	★★★★★	Monthly	Use examples from projects.
Caching	Redis, In-memory cache, Cache invalidation	★★★★★	Monthly	Mention caching for performance improvement.
Queues	Kafka / RabbitMQ basics, Message Queues	★★★	Only once	Good to mention in FairBid (for handling bidding logs).
Consistency Models	CAP theorem, Strong vs Eventual consistency	★★★★★	Monthly	Common discussion in distributed systems .
Authentication	OAuth2, JWT, Cookies vs	★★★★★	Bi-weekly	Super important in FairBid / Review

Topic	Key Concepts (must know)	Importance	Revision Frequency	Notes
	Sessions			project.
Monitoring	Logging, Metrics (basic awareness)	☆☆	Low	Just awareness is enough.

 Strategy:

- Don't deep dive → Learn by **connecting with your project**.
- Example: "For my FairBid platform, I'd use Redis cache for active bids, PostgreSQL for permanent storage, and load balancer for scaling." → That's enough.



Sheet 3: LLD (Minimal for You)

 **Goal → Only required to explain OOPS concepts in your projects.**

Concept	What to Learn	Importance	Example to Use
Class Design	Class vs Object, Constructors, Encapsulation	☆☆☆☆☆	User, Bid, Product classes in FairBid
Inheritance	Base & derived classes, Method Overriding	☆☆☆☆☆	Product → DigitalProduct vs PhysicalProduct
Polymorphism	Overloading & Overriding	☆☆☆☆☆	Payment.process(CreditCard) vs Payment.process(UPI)
Abstraction	Interfaces & Abstract Classes	☆☆☆☆☆	PaymentGateway interface with Paytm, Razorpay implementations
Design Patterns	Singleton, Factory, Observer (only 3)	☆☆☆	Singleton for DB connection, Factory for Payment, Observer for Notifications

 Strategy:

- No need to grind LLD questions.
- Just **map OOPS to your project design** → This is what interviews check.



Sheet 4: Development Stack (Next.js, React, PostgreSQL, APIs)

🎯 Goal → Be able to **design, build, and explain full-stack projects** confidently.

1. React (Frontend Basics)

Concept	Importance	Must-Know Points	Example in Your Projects
Components (Functional + Class)	★★★★★	Props, State, JSX, Component Lifecycle	FairBid: <code>BidCard</code> component
Hooks (useState, useEffect, useRef, useContext, useReducer)	★★★★★	State management, Side effects, Global state	Review Auth: track user auth state
Event Handling	★★★★★	onClick, onChange, Synthetic Events	Finance Manager: button clicks for adding transactions
Conditional Rendering	★★★★★	Ternaries, short-circuit rendering	Show verified-badge only if review is verified
Lists & Keys	★★★★★	map(), unique keys	Rendering all bids / reviews
Forms	★★★★★	Controlled vs uncontrolled components, validation	Login, review submission
React Router / Next Router	★★★★★	useNavigate / Link	Navigating between dashboard and review pages

2. Next.js (Fullstack Framework)

Concept	Importance	Must-Know Points	Example
File-based Routing	★★★★★	pages vs app directory, dynamic routes <code>[id]</code>	<code>/product/[id]</code> for FairBid
API Routes	★★★★★	<code>/api/*</code> routes for backend logic	API to submit a review
SSR vs SSG vs CSR	★★★★★	Know trade-offs	Reviews = SSR, Product listing = SSG
Data Fetching	★★★★★	getServerSideProps, getStaticProps, fetch on client	Show live bids via API

Concept	Importance	Must-Know Points	Example
Middleware	☆☆☆	For auth, request handling	Protect routes so only verified users can access
Deployment	☆☆☆☆☆	Vercel basics	Hosting your project

3. PostgreSQL (Database)

Concept	Importance	Must-Know Points	Example
Schema Design	☆☆☆☆☆	Tables, relationships (1-M, M-M), normalization	Users ↔ Reviews (1:M)
CRUD	☆☆☆☆☆	INSERT, SELECT, UPDATE, DELETE	Add new bid, fetch reviews
Indexes	☆☆☆☆	Speeds up queries	Search reviews faster
Joins	☆☆☆☆☆	INNER, LEFT, RIGHT	Fetch users with reviews
Transactions	☆☆☆☆	BEGIN, COMMIT, ROLLBACK	Payment in FairBid
Constraints	☆☆☆☆	Primary Key, Foreign Key, UNIQUE	One user can't review same course twice

4. Backend/API Basics

Concept	Importance	Must-Know Points	Example
REST APIs	☆☆☆☆☆	GET, POST, PUT, DELETE	Review system APIs
Authentication	☆☆☆☆☆	JWT, Sessions, Cookies	Verified users only
Error Handling	☆☆☆☆	Status codes, try/catch	Return 401 if not logged in
Middleware	☆☆☆☆	Logging, validation	Validate request body
Security	☆☆☆☆☆	Hashing passwords (bcrypt), input sanitization	Prevent SQL injection

5. Extra (Good to Know, Adds Interview Weight)

Concept	Importance	Must-Know Points	Example
State Management (Redux/Zustand/Context)	☆☆☆	Useful for larger projects	FairBid live bids
WebSockets (Socket.io)	☆☆☆☆☆	Real-time communication	FairBid bidding wars

Concept	Importance	Must-Know Points	Example
Caching (Redis)	★★★★	Store frequently accessed data	Recent bids in memory
Testing (Jest/React Testing Library)	★★★★	Unit + Integration tests	Testing review API

📌 How to Study:

- Don't try to *master* everything upfront → learn while **building projects**.
- Priority order: **React** → **Next.js** → **PostgreSQL** → **APIs** → **WebSockets/Redis** (as needed).
- Keep syntax sheets for React hooks + Next.js routing.
- For DB, practice with small queries daily.

✅ Interview Angle:

- When asked "*How did you build this project?*" → Explain in **layers**:
 1. **Frontend (React/Next.js)** → components, routing, SSR/CSR.
 2. **Backend (Next.js APIs)** → authentication, REST APIs.
 3. **Database (Postgres)** → schema design, constraints, transactions.
 4. **System Design** → scaling with caching, WebSockets.